Ferns - a walkthrough

Ferns

Three programs, one fern, and an algorithm that is not supposed to parallelise — until you stop trying to parallelise it and run twenty-four thousand copies of it instead.

The chaos game draws a fractal by chasing a single point through a handful of affine maps and plotting where it lands. It is *inherently sequential*: each position depends entirely on the one before it. There is no loop to split, no grid to divide, and no obvious thread.

These three examples are what happens when you take that seriously.

fern/	289 lines, 3 files	one fern, two million points, a PNG written by hand
ferns/	641 lines	a meadow growing live in a window — and structurally unable to move
fernwind/	760 lines	the same meadow on the GPU, redrawn from scratch, swaying in the wind

These documents

Chapter	What it covers
1. The chaos game	Barnsley's fern, why four affine maps *are* the plant, and the burn-in
2. One fern, one PNG	`fern/`: the sequential original, a deterministic RNG, and a PNG written by hand
3. A meadow that grows	`ferns/`: a landscape assembling live — and why it cannot be animated
4. Twenty-four thousand chaos games	`fernwind/`: the fractal-flame answer, atomics, and the probe that came first
5. The wind is in the mathematics	Rotating one map, and letting recursion turn it into a bend
6. What to understand	The things that will surprise you, and the ones that will bite

The shortest possible summary

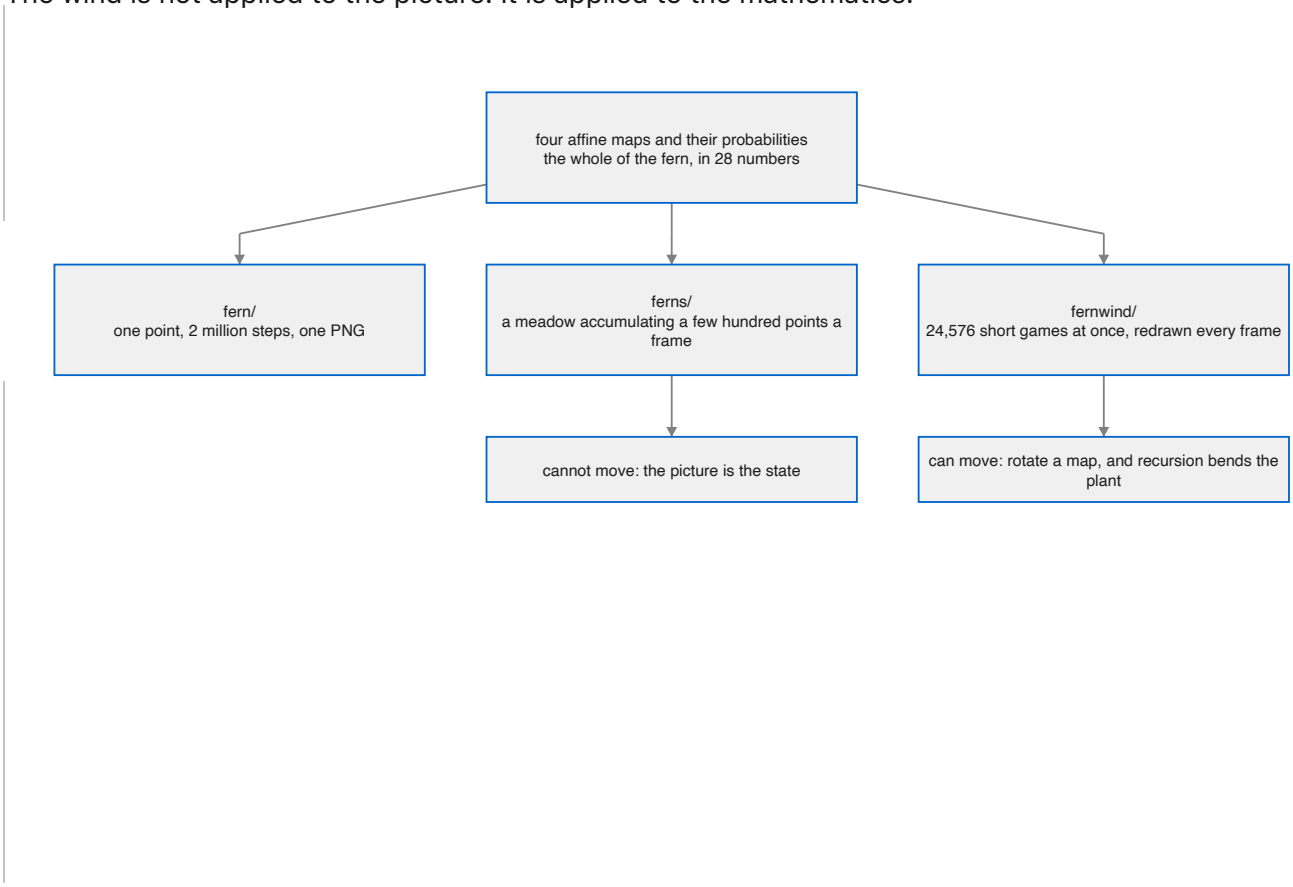
A Barnsley fern is four affine maps and their probabilities — twenty-eight numbers. Start anywhere, repeatedly pick a map at random and apply it, and the point converges onto the fern and then wanders around it forever. Plot where it goes and the fern appears. **The shape is in the numbers**, not in any drawing code.

The catch is that this is one point chasing itself. `ferns/` renders it the honest way — a few hundred points per fern per frame, accumulating into the picture — which is why that meadow **cannot move**: the picture *is* the accumulated state, and animating would mean throwing away everything drawn so far.

`fernwind/` does throw it away, every frame. Twenty-four thousand GPU threads each run their *own* short chaos game — a burn-in to reach the attractor, then a plotted stretch — and their hits meet in shared density buffers through atomic adds. Seven million points a frame, a fresh meadow each time, at 60 fps.

And redrawing from scratch is what buys the animation, because **the maps can change between frames**. Rotating each fern's climb map by a fraction of a degree bends the whole plant — because that map applies recursively, so a uniform rotation compounds into a progressive curve. Stems lean, tips whip.

The wind is not applied to the picture. It is applied to the mathematics.



1. The chaos game

Twenty-eight numbers

Here is the whole fern:

```
def barnsley() -> List[Affine]:
    """Barnsley's fern: four maps, and the shape is in the numbers."""
    var maps = List[Affine]()
    maps.append(Affine(0.00,  0.00,  0.00, 0.16, 0.0, 0.00, 0.01))
    maps.append(Affine(0.85,  0.04, -0.04, 0.85, 0.0, 1.60, 0.85))
    maps.append(Affine(0.20, -0.26,  0.23, 0.22, 0.0, 1.60, 0.07))
    maps.append(Affine(-0.15, 0.28,  0.26, 0.24, 0.0, 0.44, 0.07))
    return maps^
```

Four rows of seven numbers. There is no code anywhere in these three examples that knows what a frond is, or a stem, or a leaf. The docstring says it in seven words — **the shape is in the numbers**.

Each row is an affine map plus the probability of choosing it:

```
struct Affine(ImplicitlyCopyable, Movable):
    """x' = a x + b y + e,  y' = c x + d y + f, chosen with probability p."""
```

Read what the four are doing:

map	p	what it is
0	0.01	collapses everything onto a vertical line — **the stem**
1	0.85	shrinks by 0.85, rotates slightly, lifts by 1.6 — **the climb**
2	0.07	a squashed, rotated copy leaning right — **a frond**
3	0.07	the mirror of it — **a frond the other way**

Map 1 takes 85% of the traffic and is the one that matters most. It is a *slightly smaller, slightly rotated copy of the whole fern, moved up* — which is exactly what the plant looks like: each section is the whole thing again, smaller and turned a little. That single map builds the entire spine, and chapter 5 is about what happens when you rotate it.

The chaos game

The algorithm that turns those numbers into a picture is due to Barnsley and is startlingly simple. Start at any point. Then repeatedly:

1. pick one of the four maps at random, weighted by `p`
2. apply it
3. plot where you are

That is it. The point converges onto the fern within a few dozen steps and then wanders around it forever, visiting every part in proportion to how much of the fern is there.

There is a theorem behind it. The four maps are all **contractions** — each shrinks distances — so the system has a unique attracting set, and any starting point is drawn onto it. Iterating at random samples that set. The fern is not drawn; it is *revealed* by a point that cannot help but land on it.

```
comptime SETTLE = 20  # iterations before the point is really on the attractor
```

The first few points are **not** on the fern — they are on the way to it. Plot them and you get a short trail of stray dots leading in from wherever you started. Twenty unplotted iterations gets rid of them, and the constant is named for what it does.

`fernwind/` calls the same idea `BURN`:

```
comptime BURN = 12
...
if step < BURN:
    continue
```

> *burns in unplotted so its point is ON the attractor before anyone sees it*

Twelve rather than twenty because that version has twenty-four thousand streams, each doing its own burn-in, and the cost is paid per stream.

Why this is hard to parallelise

Look at the loop:

```
x, y  =  map[k](x, y)
```

Step *n* depends on step *n−1*, and on nothing else. There is no array to divide, no independent iterations, no reduction. It is a **recurrence** — the same structural obstacle the bifurcation example runs into with the logistic map, and the same one that makes alpha-beta a poor fit for a GPU.

You cannot split one chaos game across threads.

What you *can* do — and this is the whole of chapter 4 — is notice that the sequence's *starting point does not matter*. Every stream converges to the same attractor from anywhere. So instead of one point taking seven million steps, run twenty-four thousand points taking three hundred steps each, and let them all land on the same fern.

That is not parallelising the algorithm. It is replacing it with a different one that samples the same set.

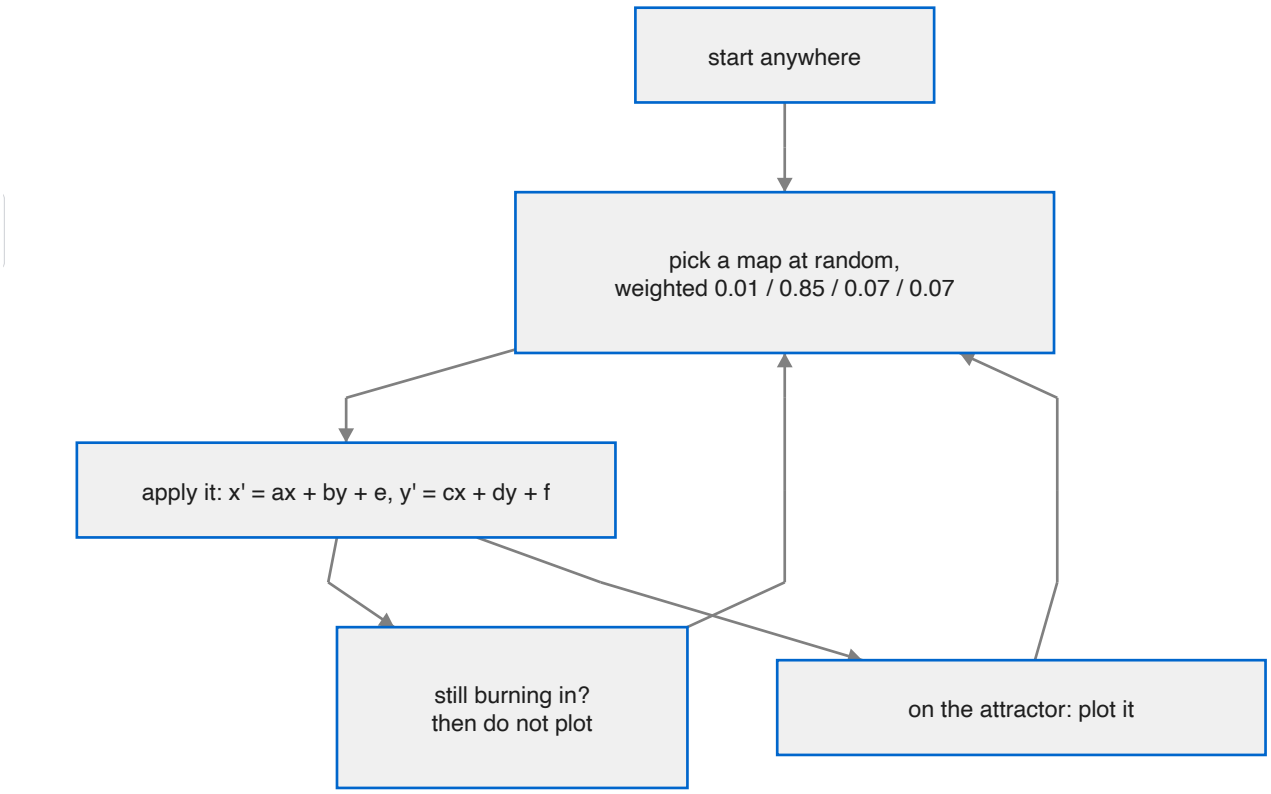
Where this comes from

Michael Barnsley set out the fern in *Fractals Everywhere* (1988), building on John Hutchinson's work on iterated function systems. The fern became the canonical demonstration for a reason that is really a compression argument: twenty-eight numbers produce an image of unbounded detail, and zooming in reveals more fern rather than more pixels.

The GPU version owes its structure to a later idea. Scott Draves's **fractal flame** algorithm (1992) took the chaos game and changed how the result is *recorded*: instead of setting pixels, accumulate a **density** — how many hits each pixel received — plus colour, and tone-map the density at the end. That turns a binary "was this pixel visited?" into a continuous measure, and it is what makes flames look lit rather than stippled.

`fernwind/` is a flame renderer with Barnsley's maps in it:

flame idea	where it is in `fernwind/`
accumulate density, not pixels	`nacc` — an atomic count per pixel
carry colour through the accumulation	`racc`, `gacc`, `bacc`, weighted by hits
tone-map at the end	the shade kernel's `n/(n+K)` coverage curve
many short streams rather than one long one	24,576 threads × 280 plotted steps



2. One fern, one PNG

`fern/` is 289 lines across three files, and it does exactly one thing: run two million steps of the chaos game and write `fern.png`.

Its structure is deliberate, and the comment says so:

```
# An iterated function system, kept apart from the example that draws it so
# there is a project here with more than one file in it.
```

The split exists partly to demonstrate the IDE's project handling — `main.mojo` imports from `ifs.mojo` and `png.mojo`, and the build follows the imports from there.

Density, not hit-or-miss

The most important decision in the file is not to plot pixels:

```
# Count how many points land in each pixel. Chaos-game pictures are all
# about density -- plotting hit-or-miss throws away most of the picture,
# which is exactly what the old text version was doing.
var hits = List[UInt32](length=W * H, fill=0)
var peak: UInt32 = 0
```

The chaos game visits parts of the fern in proportion to how much fern is there. The stem gets hammered; the tip of an outer frond gets a handful of visits in two million steps. Recording a boolean *"was this pixel ever hit?"* discards all of that, and the result is a flat green silhouette.

Counting hits keeps it. This is the same insight the fractal-flame algorithm is built on, and chapter 4 is where it becomes atomic adds in a GPU buffer.

And a log scale, for the same reason

```
# Density to colour, on a log scale: linear brightness would leave the
# fronds invisible next to the stem.
var scale = 1.0 / log(Float64(peak) + 1.0)
```

Having kept the density, you cannot display it linearly. The busiest pixel gets thousands of hits and an outer frond gets three, so a linear map paints everything except the stem black.

`log(h + 1)` normalised by `log(peak + 1)` compresses that range into something an eye can see, and the `+1` keeps `log(0)` out of it. The program prints its own busiest pixel so the number is visible:

```
print("plotted", POINTS, "points into", W, "x", H, "-- busiest pixel:", peak)
```

The colour ramp then bends each channel differently:

```
rgb.append(UInt8(24.0 + 200.0 * t * t))       # red:   t squared
rgb.append(UInt8(70.0 + 175.0 * t))           # green: linear
rgb.append(UInt8(38.0 + 150.0 * t * t * t))   # blue:  t cubed
```

Green rises fastest, red next, blue last — so sparse regions are green, dense regions warm toward white, and the fern is lit by its own density.

A deterministic RNG, on purpose

```
struct Rng(Movable):
    """A small deterministic generator, so the picture is the same every run --
    an example that draws something different each time is hard to check."""
```

xorshift64*, seeded with a fixed constant. The reasoning is about testability rather than about randomness: *an example that draws something different each time is hard to check.* Two runs produce byte-identical PNGs, so a change that alters the image is visible as a changed file.

Note the contrast with `fernwind/`, which needs the **opposite** property — each thread and each frame must differ, or the animation strobes. Same generator, opposite requirement, and both are commented where they are.

The PNG writer

`png.mojo` is 125 lines and writes a real PNG with no library:

```
# A PNG writer, small enough to read in one sitting.
#
# Truecolour, eight bits a channel, no filtering, and deflate's "stored" mode
# -- which compresses nothing but is a legal deflate stream, so every decoder
# accepts it. The whole format is four chunks and two checksums.
```

The trick worth knowing is **deflate's stored mode**. A PNG's image data must be a zlib stream, and zlib must contain deflate — but deflate has a block type meaning *"the following bytes are uncompressed"*. Emit those blocks with the right headers and you have a valid, entirely legal deflate stream that compresses nothing.

So the file is larger than it needs to be, and every PNG decoder in the world opens it. Implementing actual compression would have meant Huffman coding and LZ77 matching; this is a length, a one's-complement of the length, and the bytes.

Two checksums, both written out plainly:

```
fn _crc32(data: List[UInt8]) -> UInt32:
    """The CRC PNG puts at the end of every chunk. Bit at a time; the table
    version is faster but this one fits on the screen."""
```

> *the table version is faster but this one fits on the screen*

A 256-entry lookup table would be perhaps eight times quicker, on a function that runs four times per program. The comment records that the trade was considered and declined for legibility, which is what stops the next reader "fixing" it.

`_adler32` is zlib's own checksum, riding at the end of the compressed stream — a different algorithm at a different layer, and the file has both because the formats are nested.

The contrast this example was built for

`fern/` writes its PNG by hand. The bifurcation example does the opposite — computes in Mojo and hands the result to Python's matplotlib — and its README puts the two side by side deliberately:

> *fern/ in this collection writes a PNG by hand — 125 lines to put pixels in a file, with no axes, no ticks, no scale and no legend. That is the right answer when a bitmap is all you want. This is the other answer, and the two sit side by side on purpose.*

A fern needs no axes. A bifurcation diagram needs axes, ticks, a colour map with a legend, and a second panel sharing an x-axis — thousands of decisions somebody has already made well. The examples exist as a pair so that neither answer looks like the only one.

3. A meadow that grows

`ferns/` takes the same four maps and does something the single-fern version cannot: it shows you the chaos game *happening*.

> *Every frame each fern grows by a few hundred points, so the landscape assembles in front of you — stems first, then fronds, then the fine leaf texture as points pile up.*

That order is not staged. It is what the chaos game does: map 1 takes 85% of the traffic, so the spine appears almost immediately, and the outer fronds fill in only as the rarer maps accumulate hits. Watching it is watching the probabilities.

Depth does the design work

```
def make_fern(mut rng: Rng, base_x: Float64, base_y: Float64) -> Fern:
    """A fern for a spot on the ground. Depth does the design work: how far
    below the horizon it stands sets its size, brightness and blue-shift, so
    nearer ferns come up bigger and greener."""
    var t = (base_y - HORIZON) / (Float64(H) - HORIZON)  # 0 far .. 1 near
```

One number — how far below the horizon the fern stands — drives four things at once:

```
let scale = 5.0 + t * 16.0 + rng.next() * 2.0
let dim = 0.45 + 0.55 * t
let g_ = Int((120.0 + rng.next() * 135.0) * dim)
let b_ = Int((25.0 + rng.next() * 65.0 + (1.0 - t) * 45.0) * dim)
```

Size, brightness, and a **blue shift** that grows with distance — *"yellow-greens up close, dusty blue-greens in the distance."* That last one is aerial perspective: distant things in real air lose contrast and shift toward blue. Implementing it as `(1.0 - t) * 45.0` added to the blue channel is one term, and it is most of why the meadow reads as having depth.

There is no z-buffer, no sorting, no perspective transform. One parameter, used four times.

Everyone matures together

```
# Every fern finishes in about the same number of frames regardless of size,
# so the landscape matures together rather than the big ones dragging on.
comptime GROW_FRAMES = 600
...
# Bigger ferns need more points to fill; everyone matures together.
let target = Int(scale * scale * 380.0)
```

A fern twice as large has four times the area, so it needs roughly four times the points to reach the same visual density. `scale * scale` is that, and it means every fern reaches its target at about the same moment.

Without it the small distant ferns would finish in seconds and the near ones would still be filling in a minute later — which reads as a bug rather than as depth.

```
let delay = Int(rng.next() * 150.0)
```

And a random head start of up to 150 frames, so they do not all sprout on the same tick.

The lawn and the sky

Both are procedural, and both exist to give the ferns somewhere to be.

> *a procedural lawn — fourteen thousand individual grass blades, taller and greener up close — under a dusk sky whose clouds come from two octaves of value noise.*

The clouds:

```
# The cloud lattice: value noise, bilinearly interpolated. Coarse carries the
# cloud masses, fine breaks their edges up.
```

Two octaves rather than many: one coarse lattice for the shapes, one fine one to keep the edges from looking interpolated. Then a threshold with a smooth edge:

```
# Only the upper range of the noise is cloud; the rest stays sky.
var cloud = (n - 0.52) / 0.30
...
cloud = cloud * cloud * (3.0 - 2.0 * cloud)
cloud *= 0.75 - 0.55 * t
```

`x²(3 − 2x)` is smoothstep — it turns the hard cut at 0.52 into a soft edge. And the last line fades cloud out toward the horizon, *"where real clouds thin into haze"*: a physical observation implemented as one multiply.

Fourteen thousand grass blades are drawn individually, each with its own height, lean and shade, and there is a note about the gaps:

> *the gaps between blades read as shadow rather than void*

Which is the difference between a lawn and a field of green lines.

Why it cannot move

Here is the point of the chapter, and the reason the third example exists. From the commit that introduced the GPU version:

> *examples/ferns cannot move, and the reason is structural: the chaos game is one point chasing itself, so the picture is an accumulation and the accumulation is the state.*

Follow it through.

Each fern holds one point, and each frame advances that point a few hundred steps, plotting as it goes. The buffer accumulates. After 600 frames a fern has perhaps a hundred thousand points in it, and **those points exist only in the buffer** — there is no list of them, no scene graph, nothing to transform.

So to animate a fern you would have to either:

- **transform the accumulated pixels** — which is smearing an image, not moving a plant, and loses the density information the picture is made of; or
- **redraw from scratch each frame** — which throws away the hundred thousand points and means recomputing them all, every frame.

The second is the correct answer. It is also, on one CPU thread at a few hundred points a frame, about a thousand times too slow.

The meadow is therefore *complete* rather than *live*: it grows, it holds for five seconds, and it reseeds.

```
comptime HOLD_FRAMES = 300  # grown landscape lingers ~5 s, then reseeds
```

That is not a missing feature. It is the honest consequence of an accumulating renderer, and it is the problem chapter 4 solves — by making "redraw from scratch every frame" cheap enough to actually do.

Two doors for a harness

```
# FERNS_FRAMES=N renders N frames and exits as an unfocused Accessory, so a
# harness run never takes the screen from whoever is working. FERNS_DUMP=path
# writes the final frame as raw BGRA on the way out -- what a harness (or a
# reviewer) needs to see the landscape without a screen.
```

Two environment variables, and the first one's justification is about people rather than testing: an automated run that activates as a regular application steals focus from whoever is at the keyboard. Running as an **Accessory** means the frames render and the window never comes forward.

The same pair appears in `fernwind/` as `FERNWIND_FRAMES` and `FERNWIND_DUMP`, and in Fluid as `FLUID_AUTOSHOT`. A GUI example that cannot be checked without a person watching is one that stops being checked.

4. Twenty-four thousand chaos games

`ferns/` cannot animate because its picture *is* its state. The fix is to make the picture disposable — to rebuild the entire meadow, from nothing, sixty times a second.

That means seven million chaos-game steps per frame, which is not a thing one CPU thread does. It is, however, exactly the kind of thing a GPU does, provided you ask it the right question.

The question you cannot ask

You cannot ask a GPU to run *one* chaos game faster. Step *n* needs step *n−1*; there is nothing to divide.

The question you can

The sequence's **starting point does not matter**. Every trajectory is drawn onto the same attractor from wherever it begins, and after a short burn-in it is sampling the same fern as any other trajectory.

So: instead of one point taking seven million steps, take **24,576 points taking 292 steps each**.

```
comptime STREAMS = 24576
comptime ITERS = 280
comptime BURN = 12
comptime BLOCK = 256
comptime GRID = (STREAMS + BLOCK - 1) // BLOCK
```

```
def chaos_kernel(nacc, racc, gacc, bacc, params):
    """One thread, one short chaos game.

    The stream picks its fern by thread id, burns in unplotted so its point
    is ON the attractor before anyone sees it, then plots its stretch with
    four atomic adds per hit.
    """
```

24,576 × 280 = **6.9 million plotted points per frame**, at 60.0 fps measured.

This is not a parallelisation of the original algorithm. It is a different algorithm that samples the same set — and it works only because of the mathematical property from chapter 1: contractions have a unique attractor, and every starting point converges to it.

The burn-in is the price. Twelve steps per stream × 24,576 streams = 295,000 iterations per frame thrown away — about 4% overhead — to guarantee no stream plots a point on its way in. Skip it and the meadow acquires a haze of stray dots, one short trail per thread, 24,576 times over.

Density through atomics

```
let pat = py * W + px
_ = Atomic.fetch_add(nacc.unsafe_offset(pat), UInt32(1))
```

```
_ = Atomic.fetch_add(racc.unsafe_offset(pat), cr)
_ = Atomic.fetch_add(gacc.unsafe_offset(pat), cg)
_ = Atomic.fetch_add(bacc.unsafe_offset(pat), cb)
```

Four buffers, four atomic adds per plotted point. Twenty-four thousand threads are writing to the same 1024×640 grid with no coordination, so every write must be atomic or hits are lost.

Three of the four are colour, and the docstring says why:

> *a count, and the fern's colour weighted in, so overlapping ferns blend by evidence rather than by draw order.*

This is the fractal-flame idea. Where two ferns overlap, each contributes its colour once per hit, and the shade kernel divides by the count to get the **mean**. A pixel that got 30 hits from a blue-green distant fern and 10 from a yellow-green near one comes out three-quarters distant-fern.

There is no draw order to get wrong, because there is no drawing — only evidence accumulating, and an average taken at the end. With 24,576 threads running concurrently there *is* no meaningful order to appeal to.

Note also that the accumulators are cleared each frame. That is the whole point: the state is thrown away and rebuilt, which is what makes the maps free to change.

The probe that came first

The commit is explicit that two facts were established before anything was built on them:

> *The design rests on two facts proved by a probe before anything was built on them, since nothing in this fork's AIR path had used either:*
> - ***Atomic.fetch_add lowers through the backend without losing increments*** *(4096 threads, 4096 counted)*
> - ***host writes through map_to_host reach the next dispatch***

Both are load-bearing. If atomics dropped increments the meadow would be subtly, unfalsifiably thin — no error, just fewer points than there should be, which is indistinguishable from the tone curve being off. If host writes did not land, the per-frame parameters would be stale and the wind would not move — also with no error.

Both failures would have looked like tuning problems. A four-line probe answered each in isolation, before there was a program to blame.

That is worth generalising. This is a young backend; when a design rests on a runtime behaviour nothing in the tree has exercised, the cheapest thing you can do is exercise it on its own first. Debugging it later means debugging it through a fern.

The tone curve

```
"""Densities over the backdrop. The mean of the accumulated colour is the
```

```
fern's true shade wherever ferns overlap; coverage n/(n+K) is a curve
that density can push toward opaque but never past it, so the spine goes
solid and the wisps stay wisps and nothing blows out to white."""
```

```
let dens = Float32(Int(n))
let a = dens / (dens + Float32(3.0))
```

`n/(n+K)` with K = 3. It maps [0, ∞) into [0, 1) — one hit gives 25% coverage, three gives 50%, twenty-seven gives 90%, and no density however large reaches 1.0.

This is the same Reinhard-style curve Fluid uses on its dye, doing the same job: the fern's spine gets thousands of hits and its outer fronds get one or two, and the curve has to make both visible without the spine clipping to white.

Note the difference from `fern/`, which uses `log(h+1)/log(peak+1)`. That needs to know the peak — a second pass over the whole buffer. `n/(n+K)` needs nothing but the pixel itself, which is what a kernel with no reduction can afford. K = 3 is then tuned to the density this program actually produces:

```
# The flame: streams x plotted iterations = points per frame. On the M4 this
# is far below the frame budget; the tone curve's K is tuned to this density.
```

Change `STREAMS` or `ITERS` and the picture gets denser or thinner overall, and K has to move with it. The comment says so, which is the only thing that stops the next person raising the point count and wondering why everything went white.

Randomness that must differ twice over

```
# A stream's randomness must differ per thread AND per frame, or every
# frame replots the same points and the flame strobes.
var state = UInt32(idx) * 2654435761 ^ seed
```

Two independent requirements:

- **Per thread**, or 24,576 streams all trace the same trajectory and you get one fern's worth of points at 24,576× the density in the same places.
- **Per frame**, or every frame plots the *identical* seven million points. With enough points a fern looks complete either way — but the tiny frame-to-frame variation that makes it look alive disappears, and the result visibly strobes.

The frame seed rides in the parameter block:

```
pw[1] = Float32(frames % 8388608)
```

2^{23}, which is the largest integer a `Float32` represents exactly. The parameter block is floats, so the seed has to survive the round trip — a modulus chosen for the storage format rather than for the randomness.

This is the exact opposite of `fern/`'s requirement, where a fixed seed makes the output checkable. Same generator, opposite need, both commented.

Choosing a map in three compares

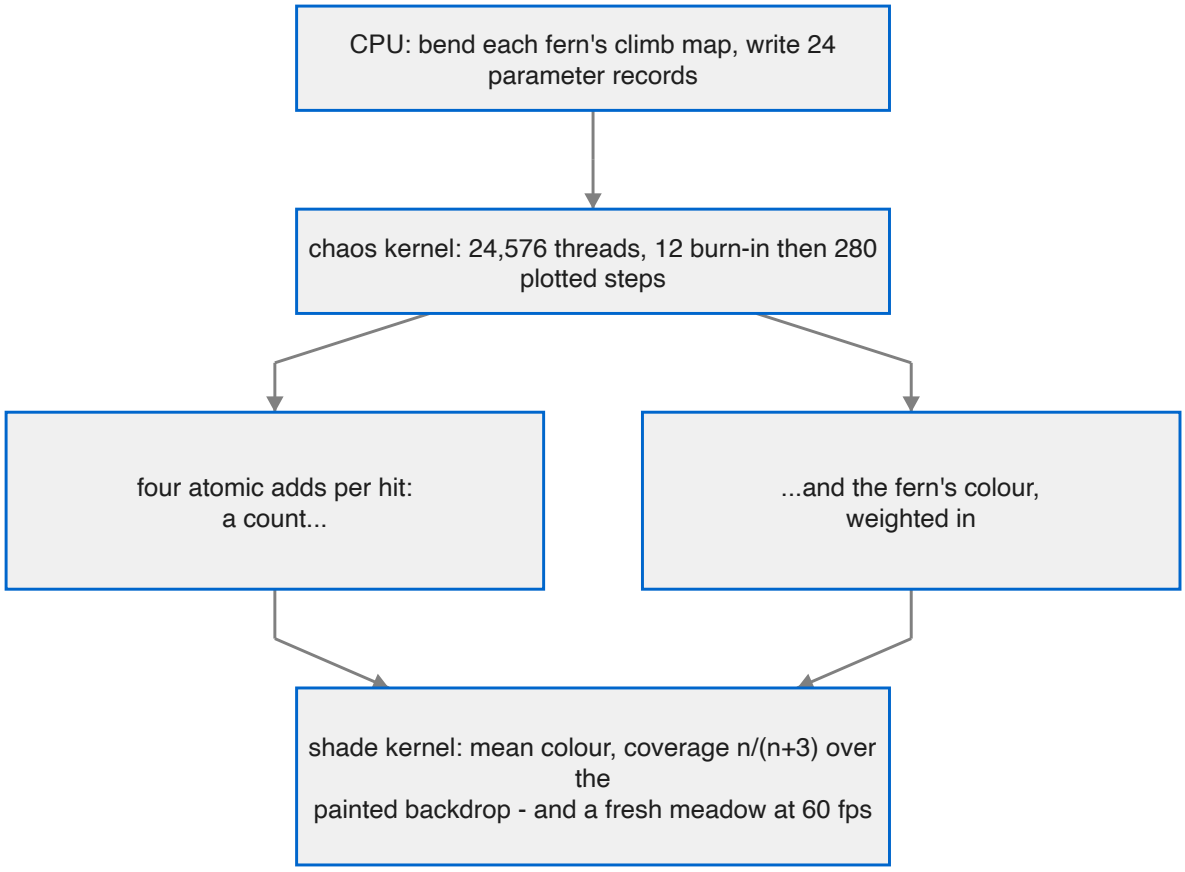
```
# Cumulative probabilities were prepared on the host, so choosing a
# map is three compares and no adds.
var m = at + 9
if roll > params[unsafe_offset = m + 6]:
    m += 7
    if roll > params[unsafe_offset = m + 6]:
        m += 7
        if roll > params[unsafe_offset = m + 6]:
            m += 7
```

`fern/` accumulates probabilities in the inner loop:

```
var acc = 0.0
for i in range(len(maps)):
    acc += maps[i].p
    if r <= acc:
```

Two million times, that is fine. Seven million times a frame on a GPU, it is four floating-point adds and a loop that could be three compares — so the cumulative sums are computed once on the CPU while the parameters are being prepared, and the kernel just walks them.

Small, and it is the kind of thing worth doing when the host is idle and the device is the bottleneck.


CPU: bend each fern's climb map, write 24 parameter records
chaos kernel: 24,576 threads, 12 burn-in then 280 plotted steps
four atomic adds per hit: a count...
...and the fern's colour, weighted in
shade kernel: mean colour, coverage n/(n+3) over the painted backdrop - and a fresh meadow at 60 fps

5. The wind is in the mathematics

This is the payoff, and it is the best idea in the three programs.

The obvious way to make a plant sway is to bend the picture of it — warp the pixels, or transform a mesh. `fernwind/` does neither. It changes **one of the four numbers-rows** before each frame, and lets the chaos game do the rest.

Rotate the climb map

Recall from chapter 1 that map 1 takes 85% of the traffic and is *a slightly smaller, slightly rotated copy of the whole fern, lifted up*. The source names it:

```
# The fern, as in fern/ifs.mojo: four affine maps, and the shape is in the
# numbers. Map 1 is the climb -- the one the wind gets to bend.
```

Before every frame, that map is composed with a small rotation:

```
if m == 1:
    # R(bend) after the climb map: linear part and
    # translation both rotate, the root stays put.
    a = bc * src.a - bs * src.c
    b = bc * src.b - bs * src.d
    c = bs * src.a + bc * src.c
    d = bs * src.b + bc * src.d
    e = bc * src.e - bs * src.f
    f = bs * src.e + bc * src.f
```

`bc` and `bs` are the cosine and sine of the bend angle. The whole map — its 2×2 linear part **and** its translation — is rotated. Rotating the translation too is what keeps the root fixed: the rotation is applied about the origin of the fern's own coordinate system, which is where the plant meets the ground.

The bend angle is a fraction of a degree.

Why a fraction of a degree bends a whole plant

Because **map 1 applies to itself, recursively**.

A point on the fern's spine has had map 1 applied to it many times over — that is what puts it up the stem. Once at the base, twice a little higher, perhaps twenty times at the tip of the topmost frond.

So a rotation composed into map 1 is applied *once* to the lowest parts and *twenty times* to the highest. The bend accumulates with height, all by itself:

> *because that map applies recursively up the plant, a uniform rotation compounds into a progressive bend: stems lean, tips whip.*

Nobody wrote "the tip should move more than the base". Nobody wrote a stiffness gradient or a chain of bones. **One uniform rotation, applied recursively, produces the correct progressive curve** — the same curve a real stem takes, for the same reason: the deflection at each point is the sum of all the deflections below it.

This only works because the fern is a *recursive* object and the animation acts on the recursion rather than on the result. Bending the image could not produce it without explicitly modelling the gradient.

The gust field

The wind itself is four sines and a phase offset:

```
let local = t - fern.phase
let gust = 0.6 + 0.4 * sin(local * 0.31 + 1.2)
let wind = gust * (
    0.5 * sin(local * 0.9)
    + 0.3 * sin(local * 2.3 + 0.8)
    + 0.2 * sin(local * 0.13)
)
```

Three sines at incommensurate frequencies — 0.9, 2.3 and 0.13 — so the sum never repeats on any timescale a viewer would notice. The slow one (0.13) provides a drift over tens of seconds; the fast one (2.3) provides the flutter.

Then `gust`, itself a slow sine between 0.6 and 1.0, multiplies the whole thing: the wind gets stronger and weaker rather than blowing evenly.

And the detail that makes it a *field* rather than a signal:

```
# Gusts travel across the meadow: each fern reads the field a
# little later the further right it stands.
let local = t - fern.phase
```

Each fern samples the same wind function at a **different time**, offset by where it stands. A gust therefore crosses the meadow rather than hitting everything at once, and the ferns are visibly at different phases — which is what a breeze across a field looks like, and it costs one subtraction.

Two effects, not one

```
let lean = fern.lean0 + wind * 0.10 * fern.supple
let bend = (wind * 0.030 + 0.010 * sin(local * 3.1)) * fern.supple * fern.flip
```

Lean is a rigid rotation of the whole fern about its base, applied when the IFS coordinates are mapped into the meadow:

```
let px = Int(base_x + fx * lean_c + fy * lean_s)
let py = Int(base_y - fy * lean_c + fx * lean_s)
```

Bend is the recursive one described above, and it carries an extra fast term — `0.010 * sin(local * 3.1)` — that the lean does not: a flutter in the fronds that the trunk does not share.

Both scale with `supple`:

```
var supple: Float64  # how much the wind moves it; taller bends more
```

A per-fern constant, so the meadow does not move as one object.

And `bend` is multiplied by `flip` as well — the mirror flag that makes half the ferns face the other way. Without it, mirrored ferns would bend *into* the wind while the others bend with it.

What the still frame shows

> *The still in review shows ferns caught mid-gust at visibly different phases.*

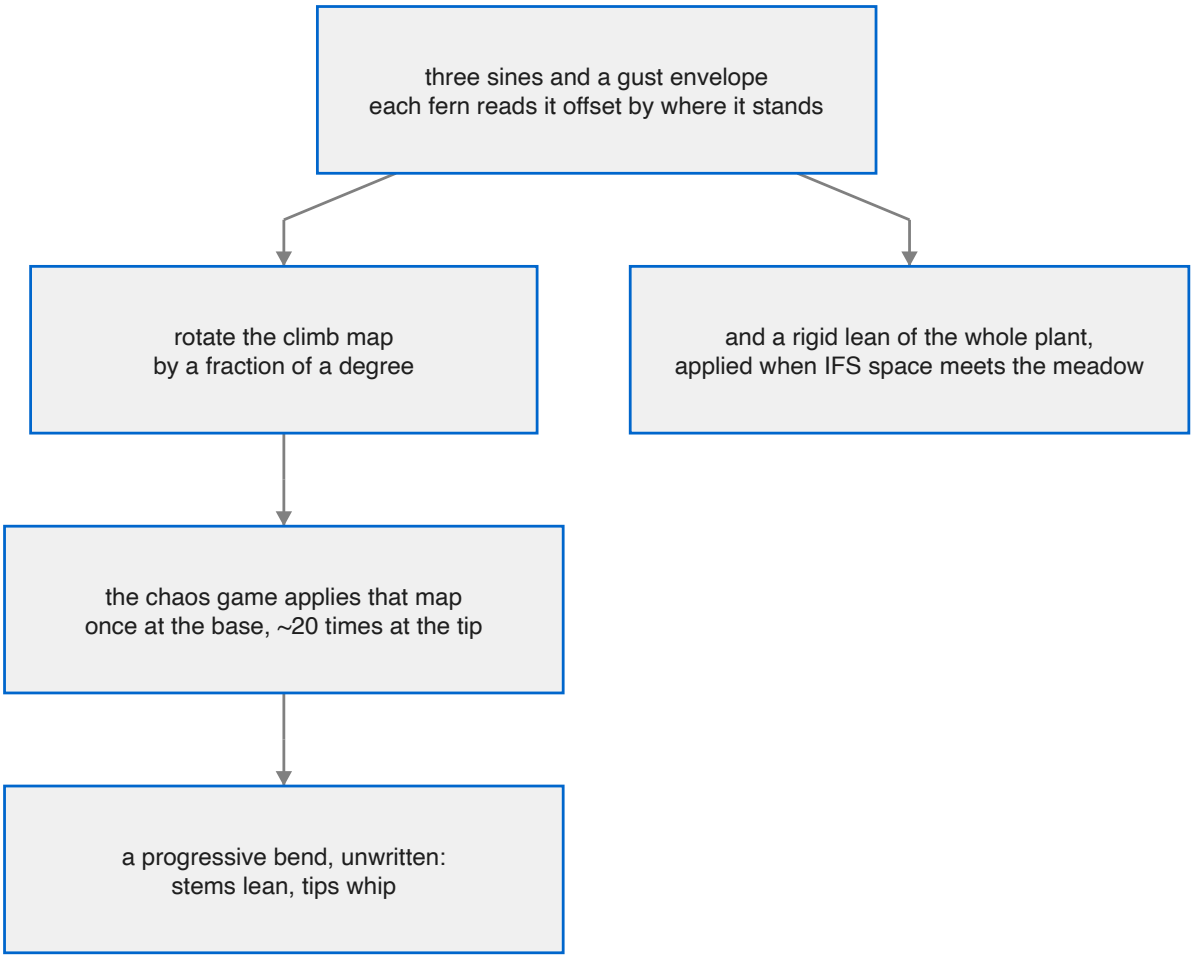
Which is the check that the phase offset is doing its job. If every fern were reading the same wind at the same instant, a single frame would show them all leaning identically — and it would look like the camera was tilted rather than like a breeze.

Why only the GPU version can do this

Everything above requires the maps to change between frames. And a fern drawn with different maps is *a different fern* — nothing about the previous frame's points can be salvaged.

So the wind requires redrawing from scratch, redrawing from scratch requires seven million points a frame, and seven million points a frame requires chapter 4.

The chain runs the other way too, and it is the neatest thing about these three programs: solving the throughput problem *for its own sake* would have produced a faster meadow. Solving it made an animated one possible, and the animation turned out to be free — one rotation, in one map, per fern, per frame.


three sines and a gust envelope
each fern reads it offset by where it stands
rotate the climb map
by a fraction of a degree
and a rigid lean of the whole plant,
applied when IFS space meets the meadow
the chaos game applies that map
once at the base, ~20 times at the tip
a progressive bend, unwritten:
stems lean, tips whip

6. What to understand

Eight things about these three programs are not obvious from watching them.

1. The fern is twenty-eight numbers

Four affine maps and their probabilities. No code anywhere knows what a stem or a frond is:

> *Barnsley's fern: four maps, and the shape is in the numbers.*

Every visual property — the spine, the curl of the fronds, the density falloff — is a consequence of those numbers and the chaos game's statistics.

2. A recurrence has nothing to divide

`x, y = map[k](x, y)` — step *n* depends on step *n−1* and nothing else. There is no loop to split and no thread to hand it to. It is the same structural obstacle the logistic map hits in bifurcation, and the same one that keeps alpha-beta off the GPU in Othello.

The answer is not to parallelise it. It is to notice that the starting point does not matter, and run 24,576 short independent games instead of one long one. A different algorithm sampling the same attractor.

3. Density is the picture; hit-or-miss throws it away

> *Chaos-game pictures are all about density — plotting hit-or-miss throws away most of the picture, which is exactly what the old text version was doing.*

The chaos game visits the fern in proportion to how much fern is there. Record a boolean and you get a flat silhouette; record a count and you get a plant.

And having kept the density you must compress it — the spine gets thousands of hits and an outer frond gets three:

`fern/`	`log(h+1) / log(peak+1)` — needs a second pass to find the peak
`fernwind/`	`n/(n+3)` — needs only the pixel, which is what a kernel can afford

4. Burn-in is not optional

```
comptime SETTLE = 20   # fern/
comptime BURN = 12     # fernwind/
```

The first points are not on the fern — they are travelling towards it. Plot them and you get a stray trail from wherever you started. Harmless once. In `fernwind/` it would be 24,576 trails, every frame, and would read as haze.

5. `ferns/` cannot move, and it is structural

> *the chaos game is one point chasing itself, so the picture is an accumulation and the accumulation is the state*

There is no list of points to transform — the hundred thousand points in a grown fern exist only as counts in a buffer. Animating means redrawing from scratch, which means recomputing everything, every frame.

That is not a missing feature. It is what an accumulating renderer *is*, and recognising it is what produced the third program.

6. Twenty-four thousand threads need twenty-four thousand dice — per frame

```
# A stream's randomness must differ per thread AND per frame, or every
# frame replots the same points and the flame strobes.
```

Two independent requirements. Share across threads and 24,576 streams trace one trajectory. Share across frames and every frame plots identical points — which still looks like a fern, and visibly strobes.

Note the frame seed is `frames % 8388608` — 2^{23}, the largest integer a `Float32` holds exactly, because the parameter block is floats.

And note that `fern/` needs the **opposite** property: a fixed seed, so *"the picture is the same every run — an example that draws something different each time is hard to check."* Same generator, opposite requirement, both commented.

7. The wind is applied to the maps, not the picture

Rotating the climb map by a fraction of a degree bends the entire plant, because that map applies **recursively** — once at the base, about twenty times at the tip:

> *because that map applies recursively up the plant, a uniform rotation compounds into a progressive bend: stems lean, tips whip*

Nobody wrote a stiffness gradient. The correct curve falls out of the recursion, and it is the same reason a real stem bends that way: the deflection at each height is the sum of everything below it.

This is only possible because the frame is rebuilt from nothing — so solving the throughput problem made the animation free rather than merely faster.

8. Probe an unproven runtime behaviour before you build on it

> *The design rests on two facts proved by a probe before anything was built on them, since nothing in this fork's AIR path had used either: Atomic.fetch_add lowers through the backend without losing increments (4096 threads, 4096 counted), and host writes through map_to_host reach the next dispatch.*

Both failures would have been silent. Dropped atomic increments give a meadow that is subtly thin — indistinguishable from the tone curve being wrong. Stale host writes give a wind that does not move — with no error either.

Two four-line probes answered both in isolation. Debugging them afterwards would have meant debugging them through a fern.

Things that will bite: drawing the fern

If you change…	…this happens
plotting to hit-or-miss	a flat silhouette; the density that *is* the picture is gone
the log or `n/(n+K)` curve to linear	everything but the spine goes black
`STREAMS` or `ITERS` without retuning K	the meadow blows out to white, or vanishes
the burn-in to zero	a stray trail per stream — 24,576 of them, every frame
the per-thread seed to a shared one	one trajectory at 24,576× density
the per-frame seed to a fixed one	identical points every frame; visible strobing
the atomic adds to plain adds	hits lost, silently, in proportion to contention

Things that will bite: moving it

If you change…	…this happens
rotating map 1's linear part but not its translation	the fern bends about the wrong point and detaches from the ground
the per-fern phase offset	the whole meadow leans as one object
dropping `flip` from the bend	mirrored ferns bend into the wind

Running them

```
cocoamojo --build examples/fern/main.mojo -I examples/fern -o /tmp/fern
```

`fern/`	writes `fern.png` beside where it was run, and prints a rough copy
`ferns/`	click plants · space pauses · `r` reseeds · `q` quits
`fernwind/`	click plants · space stills the air · `r` reseeds · `q` quits
`FERNS_FRAMES=N`, `FERNWIND_FRAMES=N`	render N frames unfocused, then exit
`FERNS_DUMP=path`, `FERNWIND_DUMP=path`	write the final frame as raw BGRA

One dialect note from building the last of them: **fn is a keyword**, and cannot be used to name a float.